DEVELOPMENT OF AN LL(1) PARSER AND PARSE TREE GENERATOR

Beginner-friendly Python implementation with FIRST, FOLLOW, LL(1) parsing table generation and parse-tree output

TEAM-8	NAME	ROLL NUMBER
1	RANJJETH	2420030569
2	MACHARLA SAI SRUJAN	2420030256
3	AKHIL	2420030315
4	DRONAVALLI SRI GOWTHAM	2420030061

Project Area: Compiler Design
Implementation Language: Python 3

1. What does this project do?

This project builds a simple LL(1) parser for a context-free grammar. It calculates FIRST and FOLLOW sets, constructs an LL(1) parsing table, checks whether the grammar has table conflicts, and parses an input token sequence. If the input is valid, it generates and displays the parse tree.

Main concepts

LL(1) parsing: A top-down parsing technique. The first L means scanning input from left to right, the second L means producing a leftmost derivation, and 1 means using one lookahead token to select a production.

FIRST: The set of terminals that can appear first in strings derived from a grammar symbol. It may contain ε when the symbol can derive an empty string.

FOLLOW: The set of terminals that can appear immediately after a non-terminal in a sentential form. The end marker $ is placed in FOLLOW of the start symbol.

Parsing table: A table that selects a production using the current non-terminal on the stack and the next input token.

Parse tree: A tree representation showing how the start symbol derives the accepted input string.

2. Project features

Feature	Purpose
Grammar input	Accepts productions with alternatives separated by |.
FIRST and FOLLOW	Computes the sets required for predictive parsing.
LL(1) table	Builds table entries from FIRST and FOLLOW.
Conflict detection	Reports collisions that indicate the grammar is not LL(1).
Input validation	Checks whether the supplied token sequence follows the grammar.
Parse-tree generation	Displays the derivation as a tree when parsing succeeds.

3. Input format and example grammar

Enter one production per line. Separate every grammar symbol with spaces. Use | for alternatives and ε for an empty production. The left-hand side of the first rule is treated as the start symbol.

```
E  -> T E'\nE' -> + T E' | ε\nT  -> F T'\nT' -> * F T' | ε\nF  -> ( E ) | id
```

Example input tokens: **id + id * id**. Tokens such as id are treated as one terminal because symbols are separated by spaces.

4. Complete Python source code

Copy the complete program below into Google Colab, IDLE or a Python 3 editor. The code is presented in one continuous listing.

```
# Development of an LL(1) Parser and Parse Tree Generator
# Grammar format: use spaces between every grammar symbol.
# Example: E -> T E'
# Use epsilon (ε) for an empty production.

def read_grammar():
    grammar = {}
    count = int(input("How many grammar rules? "))

    for _ in range(count):
        line = input("Enter rule (example E -> T E'): ").strip()
        left, right = line.split("->")
        left = left.strip()
        alternatives = [part.strip().split() for part in right.split("|")]
        grammar[left] = alternatives

    return grammar


def first_of_sequence(sequence, grammar, first):
    result = set()

    if not sequence or sequence == ["ε"]:
        result.add("ε")
        return result

    for symbol in sequence:
        if symbol not in grammar:       # Terminal
            result.add(symbol)
            return result

        result.update(first[symbol] - {"ε"})

        if "ε" not in first[symbol]:
            return result

    result.add("ε")
    return result


def calculate_first(grammar):
    first = {non_terminal: set() for non_terminal in grammar}
    changed = True

    while changed:
        changed = False

        for non_terminal, productions in grammar.items():
            for production in productions:
                before = len(first[non_terminal])
                first[non_terminal].update(
                    first_of_sequence(production, grammar, first)
                )
                if len(first[non_terminal]) != before:
                    changed = True

    return first


def calculate_follow(grammar, first, start_symbol):
    follow = {non_terminal: set() for non_terminal in grammar}
    follow[start_symbol].add("$")
    changed = True

    while changed:
        changed = False

        for left, productions in grammar.items():
```

4. Complete Python source code (continued)

```
            for production in productions:
                for i, symbol in enumerate(production):
                    if symbol not in grammar:
                        continue

                    before = len(follow[symbol])
                    remaining = production[i + 1:]

                    if remaining:
                        first_remaining = first_of_sequence(
                            remaining, grammar, first
                        )
                        follow[symbol].update(first_remaining - {"ε"})

                        if "ε" in first_remaining:
                            follow[symbol].update(follow[left])
                    else:
                        follow[symbol].update(follow[left])

                    if len(follow[symbol]) != before:
                        changed = True

    return follow


def create_parsing_table(grammar, first, follow):
    table = {non_terminal: {} for non_terminal in grammar}
    conflicts = []

    for left, productions in grammar.items():
        for production in productions:
            production_first = first_of_sequence(production, grammar, first)

            for terminal in production_first - {"ε"}:
                if terminal in table[left]:
                    conflicts.append((left, terminal))
                table[left][terminal] = production

            if "ε" in production_first:
                for terminal in follow[left]:
                    if terminal in table[left]:
                        conflicts.append((left, terminal))
                    table[left][terminal] = production

    return table, conflicts


class Node:
    def __init__(self, symbol):
        self.symbol = symbol
        self.children = []


def parse_input(tokens, grammar, table, start_symbol):
    tokens = tokens + ["$"]
    position = 0
    root = Node(start_symbol)
    stack = [(start_symbol, root)]

    while stack:
        top, current_node = stack.pop()
        lookahead = tokens[position]

        if top == "ε":
            continue

        if top not in grammar:  # Terminal
            if top == lookahead:
```

4. Complete Python source code (continued)

```
                position += 1
            else:
                print("Syntax error: expected", top,
                      "but found", lookahead)
                return None
        else:
            production = table[top].get(lookahead)

            if production is None:
                print("Syntax error: no table entry for",
                      top, "with lookahead", lookahead)
                return None

            if production == ["ε"]:
                current_node.children.append(Node("ε"))
            else:
                child_nodes = [Node(symbol) for symbol in production]
                current_node.children.extend(child_nodes)

                for symbol, child in reversed(
                    list(zip(production, child_nodes))
                ):
                    stack.append((symbol, child))

    if position == len(tokens):
        print("Input string accepted.")
        return root

    print("Syntax error: input was not fully consumed.")
    return None


def print_tree(node, level=0):
    print("  " * level + node.symbol)
    for child in node.children:
        print_tree(child, level + 1)


def show_sets(title, sets):
    print("\\n" + title)
    for symbol, values in sets.items():
        print(symbol, "=", "{" + ", ".join(sorted(values)) + "}")


def show_table(table):
    print("\\nLL(1) Parsing Table")
    for non_terminal, entries in table.items():
        for terminal, production in sorted(entries.items()):
            print("M[", non_terminal, ",", terminal, "] =",
                  non_terminal, "->", " ".join(production))


def main():
    grammar = read_grammar()
    start_symbol = next(iter(grammar))

    print("\\nOriginal Grammar:")
    for left, productions in grammar.items():
        for production in productions:
            print(left, "->", " ".join(production))

    first = calculate_first(grammar)
    follow = calculate_follow(grammar, first, start_symbol)
    show_sets("FIRST Sets:", first)
    show_sets("FOLLOW Sets:", follow)

    table, conflicts = create_parsing_table(grammar, first, follow)
    show_table(table)
```

4. Complete Python source code (continued)

```
    if conflicts:
        print("\\nGrammar is NOT LL(1). Conflicting table entries:", conflicts)
        return

    print("\\nEnter input tokens separated by spaces.")
    tokens = input("Input string (example: id + id * id): ").split()

    tree = parse_input(tokens, grammar, table, start_symbol)

    if tree:
        print("\\nGenerated Parse Tree:")
        print_tree(tree)


if __name__ == "__main__":
    main()
```

5. Example execution

Enter the following grammar when the program asks for the number of rules and productions:

```
How many grammar rules? 5\nE -> T E'\nE' -> + T E' | ε\nT -> F T'\nT' -> * F T' | ε\nF -> ( E ) | id
```

Then enter the input tokens:

```
id + id * id
```

Expected important results

```
FIRST(E)  = { (, id }\nFIRST(E') = { +, ε }\nFIRST(T)  = { (, id }\nFIRST(T') = { *, ε }\nFIRST(F)  = { (, id }\n\nFOLLOW(E)  = { $, ) }\nFOL
```

The parsing table is generated from these sets. The parser uses the table to expand non-terminals and match terminals. Since the input follows the grammar, the program prints **Input string accepted** and displays a parse tree.

6. How the parse tree is generated

1. A root node is created using the start symbol E. 2. The parser reads the next input token and checks the parsing table. 3. The selected production is used to create child nodes. 4. Child symbols are processed from left to right using a stack. 5. When all input tokens match and the end marker is reached, the input is accepted. 6. The tree is printed by visiting each node and its children.

7. How to run the project

1. Open Google Colab or Python IDLE.
2. Create a new Python file or notebook.
3. Copy the complete source code from Section 4.
4. Run the program.
5. Enter the number of grammar rules.
6. Enter each grammar production in the required format.
7. Enter the input tokens separated by spaces.
8. View the FIRST sets, FOLLOW sets, parsing table, acceptance result and parse tree.

8. Notes and limitations

• This is an educational LL(1) parser implementation intended for demonstrating compiler-design concepts.
• Grammar symbols must be separated by spaces. For example, write E -> T E' rather than E->TE'.
• The first production's left-hand non-terminal is used as the start symbol.
• The program detects parsing-table conflicts and reports that the grammar is not LL(1).
• The implementation assumes a well-formed context-free grammar and does not automatically remove left recursion or perform left factoring.
• The parser expects the input as whitespace-separated tokens, such as id + id * id.

9. Conclusion

The project demonstrates the main stages of predictive parsing: FIRST and FOLLOW computation, LL(1) parsing-table construction, syntax checking and parse-tree generation. It helps students understand how a top-down parser processes a grammar and validates an input string.